

Smart Lab Equipment Tracker & Telemetry Twin

Comprehensive System Documentation & Technical Reference Manual

1. Executive System Overview

The Smart Lab Equipment Tracker is a full-stack laboratory asset management, digital twin simulation, and predictive maintenance platform. Built with a modern React 18 frontend and Node.js Express REST API, it integrates real-time telemetry algorithms, collision-proof reservation scheduling, and automated decontamination buffers to maximize lab efficiency and prevent hardware downtime.

2. Core Feature Modules

■ AI-Powered Maintenance Predictor (/maintenance-predictor)

Analyzes multi-sensor equipment operational logs to predict breakdown risks and estimate Remaining Useful Life (RUL).

- **Telemetry Stress Vectors:** Tracks total runtime hours (10–2,500 hrs), peak operating temperature (20°C–110°C), mechanical vibration RMS (ISO 10816 standard), thermal shock cycles, and days since calibration.
- **Dual AI Engine Support:** Features a zero-latency deterministic physics engine based on thermal fatigue and wear curves, alongside an optional Groq / OpenAI LLM inference wrapper.
- **Visual Diagnostics:** Interactive circular Health Score Meter (0–100), risk badge (Low, Moderate, High, Critical), RUL countdown, root-cause diagnostics, and 1-click decontamination booking shortcut.

■ Interactive Digital Twin Simulator (/digital-twin)

A visual control panel allowing researchers and lab managers to virtually tweak operating knobs and observe simulated live telemetry waveform streams.

- **Virtual Control Panel:** Adjustable sliders for Rotor RPM (0–15,000), Temperature (4°C–95°C), Vibration Damper (0.5–5.0 mm/s), and Vacuum Chamber Pressure Integrity (20%–100%).
- **Fault Injection Suite:** One-touch stress triggers for Thermal Spike (+45°C), Rotor Imbalance, and Bearing Seizure.
- **Real-Time Visuals & Waveforms:** Animated spinning rotor dynamic schematic and HTML5 Canvas Oscilloscope plotting live multi-trace waveforms (RPM, Temperature, Vibration RMS). Includes an Emergency E-STOP interlock.

■ Smart Calendar Reservation System (/reservations)

An intelligent booking interface that eliminates double-booking conflicts and automatically enforces mandatory sterilization & decontamination windows.

- **Hourly Schedule Grid:** Visual timeline spanning operating hours (08:00 to 20:00) with color-coded booking blocks (Blue = Active Session, Purple Striped = Sterilization Buffer, Green = Available).
- **Collision & Sterilization Shield:** Enforces configurable post-session decontamination buffers (15m, 30m, 45m, 60m). Any booking overlapping a reserved slot or buffer period is automatically rejected.
- **Reservation Manifest:** Filterable booking table with instant cancellation to immediately release slots and sterilization locks.

■ Equipment Directory & Dynamic Assets (/equipment)

Comprehensive catalog featuring equipment specifications, location tracking, status tags (Available, In Use, Maintenance), dynamic search, and custom SVG visual graphics.

3. Backend REST API Specification

Endpoint	Method	Description & Functionality
/api/equipment	GET	Fetches all laboratory equipment items and baseline metadata.
/api/equipment/:id	GET	Returns full detailed record for a single equipment item.
/api/reservations	GET	Lists confirmed reservations with optional ?equipmentId= and ?date= filters.
/api/reservations	POST	Creates reservation. Validates double-bookings and sterilization buffer overlaps.
/api/reservations/:id	DELETE	Cancels booking and immediately releases reserved sterilization buffer.
/api/ai/predict-maintenance	POST	Computes equipment Health Score, RUL, stress factors, and diagnostic recommendations.

4. Backend Architecture & Resilient Datastore

The backend server features an automated zero-downtime database fallback mechanism. Upon launch, Express attempts to connect to MongoDB Atlas. If Atlas is unreachable (e.g., DNS filter or IP whitelist issue), the backend automatically initializes a high-performance in-memory datastore seeded with equipment records. The server remains online with zero downtime.

5. Setup & Running Instructions

1. Install Dependencies:

```
npm install
```

2. Start Development Environment (Frontend + Backend):

```
npm run dev
```

3. Build Production Bundle:

```
npm run --workspace frontend build
```